

WasteWise

IoT-Based Real-Time Food Waste Monitoring & Analytics System

Detailed Project Report

An initiative supported by

IIT Guwahati — Technology Innovation Hub (TIH)

Team Member	Role
Ashish Kumar Mahato	Project Mentor & Guide
Gaurav Deori	Backend, Cloud & Hardware
Nishanka Borthakur	UI/UX & Dashboard
Latifun Nisha	Research & Documentation
Parismita Sonowal	Research & Documentation

TIDF Research Building, IIT Guwahati, Assam — 781039
Repository: github.com/Gauravdeori/WasteWise

Table of Contents

1. Executive Summary.....	3
2. Problem Statement and Motivation.....	3
3. System Architecture.....	5
4. Hardware Design.....	6
5. Firmware Design.....	8
6. Cloud Layer — Multi-Channel Strategy.....	8
7. Backend Server.....	10
8. Web Dashboard.....	11
9. Security Model.....	13
10. Deployment and Operation.....	14
11. Verification and Results.....	15
12. Limitations.....	15
13. Future Work.....	16
14. Conclusion.....	16
Appendix A — Repository Structure.....	17
Appendix B — Key Parameters.....	17
Appendix C — References.....	17

1. Executive Summary

WasteWise is an end-to-end Internet-of-Things system that measures, monitors and helps reduce food waste across the 14 residential hostels of IIT Guwahati. The project replaces manual, pen-and-paper waste logging — which is slow, error-prone and rarely acted upon — with an automated pipeline in which a smart weighing scale in each hostel mess records every disposal event and streams it to a cloud service within seconds.

Each measurement node is built from low-cost, widely available components: a strain-gauge load cell, an HX711 24-bit analog-to-digital amplifier, and an ESP32 microcontroller with onboard Wi-Fi. The firmware detects a stable weight, converts it to kilograms using a calibration factor, and posts the reading — together with the hostel identifier and the board's Wi-Fi signal strength — to a ThingSpeak cloud channel over HTTPS.

A Node.js/Express server acts as a secure gateway: it holds all cloud API keys server-side, aggregates readings from four ThingSpeak channels, and exposes a small JSON API to the browser. The web dashboard, a single-page “bento grid” interface, turns those raw weights into decision-grade analytics — daily totals, monetary loss, carbon footprint, per-hostel comparison, meal-wise distribution, live device health, and a real-time feed of individual readings. Two access levels are supported: an administrator view with full operational detail and device control, and a read-only viewer view.

The system is functional and deployed. The complete data path — from a device-style write, through the cloud, to the rendered dashboard — has been verified end-to-end. This report documents the problem, the architecture, the hardware and software design, the security model, the current status, known limitations, and the roadmap.

2. Problem Statement and Motivation

2.1 The scale of food waste

Food waste is simultaneously an economic loss, a social failure and an environmental burden. The magnitude in India, drawn from the figures underpinning this project's proposal, is stark:

Indicator	Value	Source
Food wasted annually in Indian households	68.7 million tonnes	UNEP Food Waste Index
Food wasted per person per year	≈ 50 kg	UNEP Food Waste Index
India's global rank in household food wastage	2nd (after China)	UNEP Food Waste Index
Food produced that goes uneaten (US)	40 %	NRDC
Food wasted annually across Asia	≈ 1.34 billion tonnes	NRDC
Annual economic loss in India	≈ ₹89,000 crore (~1 % of GDP)	Times of Agriculture

“Studies show that businesses, such as hotels, restaurants and food retailers, earn financial returns by investing in food waste reduction.” — The Guardian

2.2 Why institutional kitchens

Institutional kitchens — hostel messes, canteens, hotels — are the highest-leverage place to intervene. They cook thousands of meals daily from a single planning decision, so a small correction in preparation quantity compounds across the entire campus. Critically, they are also measurable: waste arrives at a small

number of known disposal points at predictable times, which is exactly the condition an IoT sensing system needs.

2.3 The core gap: waste is invisible

Without measurement, food waste stays invisible. Nobody knows how much is lost, what it costs, when it peaks, or which kitchen needs attention — so no corrective action can be planned or evaluated. Manual registers, where they exist, are inconsistent, delayed and never aggregated. WasteWise addresses this gap directly: it makes waste continuously visible, quantified in kilograms, rupees and kilograms of CO₂e, and attributable to a specific hostel and meal.

2.4 Objectives

Vision — to create a milestone “Zero Food Waste Institute” in India and establish a replicable model for campuses, hostels, hotels and restaurants nationwide.

Mission — to record, track and reduce food waste easily and reliably using IoT, replacing pen and paper with automated real-time monitoring that makes waste visible, measurable and actionable.

Specific objectives:

- Automate food-waste recording at source using an IoT waste scale, eliminating manual entry.
- Visualise waste data in real time through a cloud-connected dashboard.
- Centralise records for all hostels on one platform to enable comparison and analysis.
- Quantify the amount, monetary value and carbon footprint of waste.
- Support data-driven optimisation of food production and menu planning.

3. System Architecture

3.1 Design philosophy

The architecture is deliberately decoupled into four layers, each replaceable without disturbing the others. Two constraints drove the design: the hardware node must be inexpensive enough to replicate fourteen times (and later, campus-wide), and no cloud credential may ever reach the end user's browser.

A notable consequence of this separation is that the measurement devices do not depend on the WasteWise server at all. Each ESP32 publishes directly to ThingSpeak, so readings continue to be captured and stored even if the dashboard or its server is offline; the server is required only to view and analyse the data, never to collect it.

3.2 End-to-end data flow

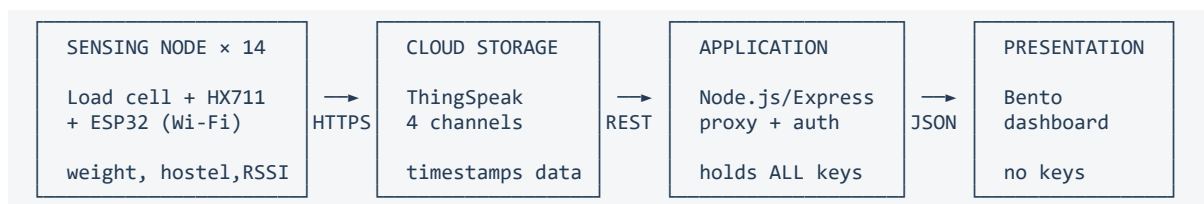


Figure 1 — Four-layer architecture. Devices write to the cloud independently of the server.

3.3 Layer responsibilities

Layer	Technology	Responsibility
Sensing	Load cell, HX711, ESP32 (Arduino C++)	Detect a stable disposal event, convert raw counts to kg, publish over Wi-Fi
Cloud storage	ThingSpeak (4 channels)	Timestamp, persist and serve time-series readings; decouples device from server
Application	Node.js 18+, Express 4, dotenv	Hold credentials, aggregate channels, authenticate admins, expose JSON API, serve site
Presentation	HTML/CSS/vanilla JS, inline SVG	Compute analytics client-side, render bento dashboard, enforce role-based views

4. Hardware Design

4.1 Component selection

Component	Function	Rationale
ESP32 Dev Module	Microcontroller + Wi-Fi radio	Integrated 2.4 GHz Wi-Fi and HTTPS stack; no external network module needed
HX711 amplifier	24-bit ADC for the load cell	Purpose-built bridge amplifier; resolves the microvolt-level signal a load cell produces
Strain-gauge load cell	Converts force to a voltage	Standard, low-cost, mechanically simple; adequate accuracy for kitchen-scale waste
Waste bin / platform	Receives the disposal	Fits existing mess workflow — no behaviour change required from students

A load cell alone cannot be read by a microcontroller: its output is a differential signal of a few millivolts at full scale. The HX711 solves this with a high-gain, low-noise 24-bit converter that communicates over a simple two-wire protocol, which is why this pairing is the standard approach for digital scales.

4.2 Wiring

HX711 pin	ESP32 connection	Purpose
VCC	3V3	Power supply
GND	GND	Common ground
DT (DOUT)	GPIO 16	Serial data out from the ADC
SCK (PD_SCK)	GPIO 4	Clock driven by the ESP32

The load cell connects to the HX711 bridge inputs using the conventional colour code: E+ / E- (excitation) to red / black, and A+ / A- (signal) to white / green.

4.3 Calibration

A load cell reports raw ADC counts, not mass; the mapping between the two depends on the specific cell, its mounting and the excitation. Calibration establishes that constant, and it is the single most error-prone step in bringing up a scale. The firmware therefore includes a dedicated calibration mode:

1. Set `CALIBRATION_MODE = true` and upload; open the Serial Monitor at 115200 baud.
2. Leave the platform empty — the firmware automatically tares (zeroes) the scale.
3. Place an object of accurately known mass (for example 500 g).
4. Read the printed raw count and compute the constant: $\text{CALIBRATION_FACTOR} = \text{raw} \div \text{known_grams}$.
5. Enter that value, set `CALIBRATION_MODE = false`, and re-upload. Readings are now true kilograms.

Worked example: a raw reading of 210,000 counts under a 500 g reference gives a calibration factor of $210000 \div 500 = 420.0$.

The firmware additionally performs a wiring self-check at boot. If the HX711 does not respond within 1.5 s the console prints an explicit diagnostic naming the pins to inspect, which distinguishes a wiring fault from a calibration fault — historically the most common source of confusion during deployment.

5. Firmware Design

5.1 Event-driven measurement

The scale does not stream continuously; that would flood the cloud, exceed rate limits and produce meaningless data. Instead it recognises a discrete disposal event through a small state machine:

6. Idle — the platform is empty and the firmware polls the load cell about five times per second.
7. Detection — a mass above MIN_EVENT_KG (0.05 kg) appears, rejecting vibration and noise.
8. Stabilisation — the reading must stay within STABLE_TOL_KG (0.02 kg) for STABLE_SAMPLES (8) consecutive samples, so the value is captured only after the load settles. A 6-second timeout prevents an indefinite wait.
9. Transmission — the stabilised weight is posted to ThingSpeak with the hostel code and Wi-Fi RSSI.
10. Lockout — no further reading is sent until the platform is cleared, guaranteeing one record per disposal.

This design yields clean, human-meaningful records: one row per act of throwing food away, rather than a noisy continuous signal that would have to be de-duplicated later.

5.2 Transmitted payload

Field	Content	Units	Purpose
field1	Stabilised weight	kg	The primary measurement
field2	Hostel code (1–14)	integer	Attributes the reading to a hostel; drives all per-hostel analytics
field3	Wi-Fi RSSI	dBm	Device health — surfaces link quality in the dashboard's Device Monitor

Including RSSI turns each data point into a health check as well as a measurement. A node whose signal is degrading can be identified and relocated before it starts dropping readings — a practical necessity when fourteen nodes are distributed across a large campus.

5.3 Rate-limit compliance

The ThingSpeak free tier accepts at most one update per channel every 15 seconds. The firmware enforces a 16-second minimum interval (TS_MIN_INTERVAL_MS) and, if an event arrives sooner, waits rather than allowing a silent rejection. The console reports the wait explicitly, and a "0" response from ThingSpeak — its signal for a rejected write — is reported with a hint naming the two likely causes (bad key or rate limit).

6. Cloud Layer — Multi-Channel Strategy

ThingSpeak was selected as the cloud service on the project's recommendation. It provides timestamped time-series storage, a REST API and a free tier sufficient for a pilot, with no server administration.

6.1 Why four channels

A single channel would have been a hard bottleneck. With the free tier's 15-second write limit, fourteen hostels sharing one channel would contend for the same slot: during a lunch rush several messes dispose

of waste simultaneously, and every collision would be silently rejected — precisely the readings that matter most. Distributing the hostels across four channels multiplies the aggregate write capacity roughly fourfold and confines any contention to a small group.

6.2 Hostel-to-channel mapping

Channel	ThingSpeak ID	Hostel codes	Hostels
Channel 1	3427253	1 – 4	Brahmaputra, Subansiri, Lohit, Dhansiri (Girls)
Channel 2	3427254	5 – 8	Manas, Kopili, Barak, Jia Bharali
Channel 3	3427424	9 – 11	Pagladiya, Dikhow, Dhansiri (Boys)
Channel 4	3425567	12 – 14	Kojoli Annex, Bhrigu, Neelachal

The mapping is implemented symmetrically at both ends. In the firmware, `getWriteApiKey(stationCode)` returns the correct write key for the board's hostel code, so each node targets its own channel automatically. On the server, `getChannelIndex(hostelCode)` performs the inverse mapping for writes, while reads fan out across all four channels concurrently. Adding a hostel therefore requires changing only these two mappings.

7. Backend Server

The backend (server.js, ~300 lines, Express 4 on Node 18+) has three jobs: keep secrets, aggregate the cloud channels, and gate privileged actions. It also serves the static front-end, so a single process runs the whole application.

7.1 API surface

Endpoint	Method	Auth	Description
/api/config	GET	public	Non-secret state: channel id, whether writes are enabled, whether admin auth is configured
/api/feeds	GET	public	Merged, time-sorted readings from all four channels; applies the reset filter
/api/login	POST	public	Validates admin credentials server-side; returns an admin token on success, 401 otherwise
/api/reading	POST	admin token	Writes a reading, routed to the channel that owns the given hostel
/api/reset	POST	admin token	Clears the dashboard history (and the cloud channels when a User API Key is present)

7.2 Channel aggregation

Because the fourteen hostels are spread over four channels, /api/feeds issues four HTTPS requests concurrently with Promise.all, concatenates the responses, sorts the union chronologically, applies the reset filter, and truncates to the requested number of most-recent records. Fetching in parallel rather than sequentially keeps dashboard latency close to that of a single channel; each request carries a 10-second timeout so one slow channel cannot hang the response. The client receives a single, uniform feed and remains entirely unaware of the sharding — which is what allows the number of channels to change later without touching the front-end.

7.3 Reset semantics

Clearing feed data from a ThingSpeak channel requires the account-level User API Key, which is deliberately more privileged than the per-channel read/write keys. The reset endpoint therefore implements two tiers:

- Soft reset (always available) — the server records the reset instant in reset_timestamp.txt and thereafter filters out every reading older than that moment. The dashboard starts from a clean slate immediately, and because the marker is a file it survives a restart. The historical data itself remains in the cloud, so the action is recoverable by deleting the marker.
- Hard reset (requires TS_USER_API_KEY) — the server additionally issues an HTTP DELETE to each of the four channels, permanently erasing the stored feeds. This is irreversible.

The endpoint reports honestly which tier actually executed, so an operator is never left believing the cloud was wiped when only the local view was reset. The two-tier approach is a pragmatic response to a real constraint: the destructive capability is optional and opt-in, while the everyday need — a clean demonstration — is always satisfied without risk.

8. Web Dashboard

The dashboard is a single-page application written in vanilla HTML, CSS and JavaScript with no front-end framework and no build step; all charts are hand-generated inline SVG. This keeps the deployable artefact to a handful of static files, removes an entire class of toolchain maintenance, and makes the project readable by future contributors without framework expertise. It polls the backend every 20 seconds (REFRESH_SECONDS).

8.1 Layout

The interface uses a “bento grid”: a status bar, two vertical rails listing all fourteen hostels, and a central analytics column. Every hostel card shows today’s weight, an efficiency score, a seven-day sparkline and a live/offline status, and is clickable — selecting one re-scopes the entire centre column to that hostel and back again via an “Overview” control.

Panel	Content
Status bar	Live meal session, running clock, active-device count (real ESP32s counted separately), alert bell, admin controls
Hostel rails	14 hostel cards: today’s kg, efficiency %, 7-day sparkline, live/offline state; auto-scrolling in overview mode
KPI tiles	Total waste today and estimated value lost, each with a day-on-day delta
Waste Trend	Area chart over a selectable 7 / 14 / 30-day window with per-day markers and a hover tooltip
Live Readings	Real-time feed — every individual reading with weight, hostel, meal and timestamp; new rows highlight
Waste by Meal	Donut showing the breakfast / lunch / snacks / dinner split for the week
Live Scale Readings	Per-bin current totals and online state (admin only)
Insight tiles	Best-performing hostel, peak waste time, estimated CO ₂ saved
Admin analytics	Hostel comparison bar chart and cost analysis (today, week, monthly projection)
Device Monitor	Connected ESP32s with last-seen time and Wi-Fi signal strength (admin only)
Recommendation	Rule-based advice derived from today’s data versus the 7-day average (admin only)
Alerts	High-waste and offline-device notifications (admin only)

8.2 Derived metrics

The dashboard converts raw mass into the units in which decisions are actually made. All factors are declared in config.js and can be tuned without touching code:

Metric	Formula	Constant
Value lost (₹)	$\text{weight_kg} \times \text{PRICE_PER_KG}$	₹75 per kg
Carbon footprint (kg CO ₂ e)	$\text{weight_kg} \times \text{CO2_PER_KG}$	2.5 kg CO ₂ e per kg
Per-capita waste (g)	$\text{weight_kg} \times 1000 \div \text{residents in scope}$	450 residents per hostel
Meals equivalent	$\text{weight_kg} \div \text{KG_PER_MEAL}$	0.4 kg per meal
Efficiency score (%)	$\text{clamp}(100 - \text{per-capita grams} \div 1.6, 40, 99)$	derived
“High waste” flag	$\text{today} > 1.3 \times \text{7-day average}$	derived

Interpretation note. The monetary and carbon figures are planning estimates produced by a single multiplier, not audited measurements — their purpose is to express waste in units a warden or caterer can act on. The efficiency score is likewise a normalised internal indicator for ranking hostels, not a physical quantity. Only the weight, the timestamp, the hostel and the RSSI are directly measured.

8.3 Data integrity

A miscalibrated or faulty load cell can emit implausible values, and a single wild reading would visibly distort every aggregate on the page. The client therefore discards any reading outside MIN_VALID_KG (0.02 kg) to MAX_VALID_KG (60 kg) before it enters the statistics. This bound is a safety net for sensor faults, not a substitute for calibration.

A DEMO_FILL switch allows hostels without a physical device to be populated with deterministic simulated data so the full interface can be demonstrated before hardware rollout completes. It is currently set to false, so the dashboard presents real measurements only, and hostels sending genuine data are visually distinguished from simulated ones.

9. Security Model

9.1 Credential isolation

The system originally embedded ThingSpeak keys directly in client-side JavaScript, which meant every visitor received working cloud credentials. This was corrected by introducing the backend proxy. All keys now reside in a .env file that is excluded from version control and never served — the static handler explicitly denies dotfiles, so neither .env nor .git is reachable over HTTP. The browser holds no cloud credential of any kind and can only reach ThingSpeak through the server's narrow API.

9.2 Roles and write protection

Role	Verification	Capabilities
Administrator	Server-side (/api/login against .env)	Full dashboard, device monitor, alerts, recommendations, CSV export, manual logging, data reset
Viewer	Client-side (read-only account)	Simplified read-only dashboard: KPIs, trend, meal split, insights, hostel rails

Privileged actions are enforced on the server, not merely hidden in the interface. /api/reading and /api/reset are wrapped in a requireAdmin middleware that demands a valid token in the x-admin-token header and returns HTTP 403 otherwise. This was verified by calling the write endpoint directly with no token and as a signed-in viewer; both were rejected by the server. Hiding a button is a usability measure — the middleware is the actual control.

9.3 Open issues (honest assessment)

The following are genuine weaknesses in the current build and should be resolved before any public or production deployment.

- Admin credentials in client configuration. To make login work on the hosted deployment without server environment variables, an admin entry — including the password and an admin token — was reintroduced into config.js, which ships to every browser. The committed copy currently carries a placeholder token ("demo-token"), so the live secret is not yet public, but the admin password is, and the working copy now contains the real token. Committing it in this state would publish a credential that grants write and reset access.
- Credential history. Cloud keys were committed early in development. The repository history was subsequently rewritten with git filter-repo to purge them, but keys that were ever public should be considered compromised and regenerated on ThingSpeak.
- Static shared token. The admin token is a fixed secret rather than a per-session, expiring credential. It cannot be revoked individually or timed out.
- Viewer authentication is cosmetic. The viewer account is checked in the browser and protects nothing; read endpoints are unauthenticated by design.

Recommended remediation: remove the admin password and token from config.js and restore server-only verification by configuring ADMIN_USER, ADMIN_PASS and ADMIN_TOKEN as environment variables in the hosting platform; regenerate every ThingSpeak key; and, for production, replace the static token with signed, expiring sessions.

10. Deployment and Operation

10.1 Running locally

```
cd "TIH IOT Project"
cp .env.example .env      # then fill in ThingSpeak keys + admin credentials
npm install               # express, dotenv
npm start                 # → http://localhost:3000
```

10.2 Hosting

The server exports the Express app and only binds a port when executed directly, which allows the same file to run as a long-lived Node process locally and as a serverless function on Vercel (configured in `vercel.json`). Because credentials live in the environment rather than the repository, every deployment target must define them independently — the absence of these variables is the usual cause of a deployed instance returning HTTP 500 from `/api/feeds` while the local instance works.

Variable	Purpose	Required
TS_CHANNEL_1..4_ID	ThingSpeak channel identifiers	Yes (multi-channel)
TS_READ_KEY_1..4	Per-channel read keys	Yes
TS_WRITE_KEY_1..4	Per-channel write keys	For manual logging
ADMIN_USER / ADMIN_PASS	Admin credentials, verified server-side	For admin login
ADMIN_TOKEN	Secret required by write endpoints	For admin actions
TS_USER_API_KEY	Account key enabling a hard channel wipe	Optional

11. Verification and Results

The following behaviours were exercised directly against the running system rather than assumed:

Test	Method	Result
End-to-end data path	Device-style write to ThingSpeak, then read back through /api/feeds	Weight, hostel and RSSI returned intact and rendered
Multi-channel aggregation	Concurrent fetch of 4 channels, merged and sorted	Single unified feed returned to the client
Write authorisation	POST /api/reading with no token	HTTP 403 — rejected
Admin login	POST /api/login with wrong then correct password	401, then token issued
Viewer cannot write	Forced fetch to /api/reading while signed in as viewer	HTTP 403 — rejected by server
Role-based interface	Sign in as viewer and as admin	Admin-only panels hidden for viewer; restored for admin
Reset authorisation	POST /api/reset with no token	HTTP 403 — rejected
Secret exposure	Request /.env over HTTP	Not served
Credential isolation	Inspect browser network traffic	No request to ThingSpeak; only same-origin /api/* calls

11.1 Current status

The software platform is complete and operational: the cloud channels are provisioned, the multi-channel backend aggregates them, and the dashboard renders live readings with role separation. Firmware is complete and includes calibration and self-check tooling. The remaining work is physical rollout — calibrating and installing a node in each of the fourteen hostels — together with the security remediation set out in section 9.3.

Observed during testing: a day of test traffic produced an aggregate far above the surrounding days, traced to individual readings of implausible magnitude for a single disposal. This is the signature of an uncalibrated load cell rather than a software fault, and it illustrates why per-reading visibility (the Live Readings feed) and the validity bounds were added — an aggregate alone would have concealed the cause.

12. Limitations

- No category sensing. A load cell measures mass only; it cannot distinguish rice from vegetables. Category analytics would require an operator selection at the bin or an added sensing modality.
- Cloud rate limits. The free tier permits one write per channel per 15 seconds. Four channels ease this, but a campus-wide rollout would need a paid tier or self-hosted ingestion.
- Calibration is per-unit and manual. Each scale must be calibrated individually, and drift or mechanical disturbance requires recalibration.
- Wi-Fi dependence. A node with no network cannot deliver readings; the firmware retries but does not yet buffer offline events to local storage.
- Estimated economics. Monetary and carbon values use fixed campus-wide multipliers and are indicative, not audited.
- Demonstration-grade authentication, as detailed in section 9.3.

13. Future Work

Priority	Item	Rationale
Immediate	Remove admin credentials from config.js; set server environment variables	Closes the live credential exposure
Immediate	Regenerate all ThingSpeak keys	Previously-public keys must be treated as compromised
Immediate	Calibrate and deploy the 14 nodes; set DEMO_FILL = false	Completes the pilot with real data campus-wide
Short term	Offline buffering in firmware	Prevents data loss during network outages
Short term	Physical tare button and enclosure	Field robustness and easy re-zeroing between meals
Short term	Threshold alerts by email/SMS	Pushes action to wardens without dashboard monitoring
Medium term	Signed, expiring sessions and per-user accounts	Production-grade access control and auditability
Medium term	Category capture at the bin	Enables menu-level, not just quantity-level, optimisation
Long term	Predictive preparation guidance	Forecast demand from history to prevent waste at source
Long term	Extension beyond campus	Realises the replicable model for hotels and restaurants

14. Conclusion

WasteWise demonstrates that the invisibility of institutional food waste is an addressable engineering problem. Using inexpensive, standard components — a load cell, an HX711 amplifier and an ESP32 — the system captures a clean, attributable record of every disposal and turns it, within seconds, into figures a decision-maker can act on: kilograms today, rupees lost, carbon emitted, which hostel is worst, and when the peak occurs.

The engineering emphasis has been on correctness and honesty rather than surface polish. Devices publish independently of the server, so measurement survives application downtime. Cloud credentials were moved out of the browser and privileged operations are enforced by server-side middleware, verified by direct testing rather than assumed. Implausible sensor values are bounded, estimates are labelled as estimates, and the destructive reset defaults to a recoverable soft reset. Where weaknesses remain — most importantly the admin credentials currently present in client configuration — they are documented in section 9.3 rather than obscured.

With the software platform complete and verified, the path to the project's stated vision — a Zero Food Waste Institute, and a model replicable across campuses, hotels and restaurants — now runs through physical deployment: calibrating and installing the fourteen nodes, closing the security gaps, and beginning to feed the resulting evidence back into menu and production planning.

Appendix A — Repository Structure

TIH IOT Project/	
— server.js	Express backend: proxy, auth, multi-channel merge, reset
— package.json	Dependencies (express, dotenv); Node >= 18
— vercel.json	Serverless deployment configuration
— .env	Secrets – gitignored, never served
— .env.example	Template for required environment variables
— index.html	Landing page + dashboard markup
— style.css	All styling (bento dashboard, responsive rules)
— script.js	Site navigation and animations
— dashboard.js	Dashboard logic: auth, analytics, SVG charts, rendering
— config.js	Front-end settings: hostels, factors, thresholds
— reset_timestamp.txt	Soft-reset marker consumed by the feeds endpoint
— assets/	Images
— firmware/esp32_foodwatch/esp32_foodwatch.ino	ESP32 + HX711 firmware

Appendix B — Key Parameters

Parameter	Value	Location	Meaning
MIN_EVENT_KG	0.05 kg	firmware	Minimum mass treated as a disposal event
STABLE_TOL_KG	0.02 kg	firmware	Tolerance band defining a settled reading
STABLE_SAMPLES	8	firmware	Consecutive stable samples required
TS_MIN_INTERVAL_MS	16000 ms	firmware	Enforced gap between cloud writes
CALIBRATION_FACTOR	per unit	firmware	Raw ADC counts per gram
MIN_VALID_KG / MAX_VALID_KG	0.02 / 60 kg	config.js	Validity bounds for an accepted reading
PRICE_PER_KG	₹75	config.js	Monetary value of 1 kg of food
CO2_PER_KG	2.5 kg CO ₂ e	config.js	Emissions per kg of wasted food
KG_PER_MEAL	0.4 kg	config.js	Average mass of one meal
HOSTEL_POPULATION	450	config.js	Residents per hostel, for per-capita figures
REFRESH_SECONDS	20 s	config.js	Dashboard polling interval
DEVICE_ONLINE_MINUTES	10 min	config.js	Recency window defining an online device
DEMO_FILL	false	config.js	Simulated data disabled — real readings only

Appendix C — References

- United Nations Environment Programme (UNEP), Food Waste Index Report — household food waste in India.
- National Resources Defense Council (NRDC) — food loss and waste in the United States and Asia.
- Times of Agriculture, “Food Wastage in India: From Farm to Bin” — economic loss estimates.
- The Guardian — financial returns from business investment in food-waste reduction.
- MathWorks ThingSpeak REST API documentation — channel read, update and clear operations.
- B. Necula, HX711 Arduino Library (bogde/HX711).
- Espressif Systems, ESP32 Arduino Core documentation.

